ವಿಶ್ವೇಶ್ವರಯ್ಯ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ, ಬೆಳಗಾವಿ
VISVESVARAYA TECHNOLOGICAL UNIVERSITY - BELAGAVI

BANGALORE INSTITUTE OF TECHNOLOGY

K.R. ROAD, V.V PURAM, BENGALURU – 560 004

INFORMATION SCIENCE AND ENGINEERING

(AFFILIATED TO VTU, BELAGAVI)

CODE: BCS702

PARALLEL COMPUTING LAB

As per Choice Based Credit System Scheme (CBCS) FOR

VII SEMESTER CSE/ISE AS PRESCRIBED BY VTU

Prepared By:

Prof. Vani V
Associate Professor
Dept of ISE, BIT

BANGALORE INSTITUTE OF TECHNOLOGY

VISION:

Establish and develop the Institute as the Centre of higher learning, ever abreast with expanding horizon of knowledge in the field of Engineering and Technology with entrepreneurial thinking, leadership excellence for life-long success and solve societal problems.

MISSION:

- Provide high quality education in the Engineering disciplines from the undergraduate through doctoral levels with creative academic and professional programs.
- Develop the Institute as a leader in Science, Engineering, Technology, Management and Research and apply knowledge for the benefit of society.
- Establish mutual beneficial partnerships with Industry, Alumni, Local, State and Central Governments by Public Service Assistance and Collaborative Research.
- Inculcate personality development through sports, cultural and extracurricular activities and engage in social, economic and professional challenges.

Bangalore Institute of Technology

K. R. Road, V. V. Pura, Bengaluru- 560004

Department of Information Science and Engineering

VISION:

Empower every student to be innovative, creative and productive in the field of Information Technology by imparting quality technical education, developing skills and inculcating human values.

MISSION:

M1	To evolve continually as a Centre of Excellence in offering quality Information Technology **Education**.
M2	To nurture the students to meet the global competency in industry for **Employment**.
M3	To promote collaboration with industry and academia for constructive interaction to empower **Entrepreneurship.**
M4	To provide reliable, contemporary and integrated technology to support and facilitate **Teaching, Learning, Research and Service.**

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO-1	Uplift the students through Information Technology **Education.**
PEO-2	Provide exposure to emerging technologies and train them to **Employable** in multi-disciplinary industries.
PEO-3	Motivate them to become good professional Engineers and **Entrepreneur.**
PEO-4	Inspire them to prepare for **Higher Learning and Research.**

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO-1	To provide our graduates **with Core Competence in Information Processing and Management.**
PSO-2	To provide our graduates with Higher Learning in **Computing Skills.**

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Bangalore Institute of Technology

K. R. Road, VV Pura, Bangalore 560004

Department of Information Science and Engineering

Parallel Computing Laboratory (BCS702)

PREREQUISITE:

1. Knowledge of C programming
2. Operating System
3. Data Structures and Algorithms
4. Computer Organization and Architecture

COURSE OUTCOMES:

VII-SEM Course Name: Parallel Computing Course Code: BCS702 Year of Study: 2025-26	
At the end of the course the students should be able to:	
CO1	**Understand** parallel programming concepts like classifications of parallel computers, various architecture, memory models and tools.
CO2	**Apply** parallel programming paradigms using MPI, OpenMP, and CUDA to solve computational problems.
CO3	Analyze the performance of parallel programs by evaluating various performance metrics.
CO4	Design and implement solutions for real time problems by integrating parallel programming techniques.

Mapping of COs-POs and COs-PSOs

CO's with PO's and PSO's Mapping														
VII-SEM Course Name: Parallel Computing Course Code: BCS702 Year of Study: 2025-26														
Cos	**PO1**	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PO12**	**PSO1**	**PSO2**
CO1	2													
CO2	3												2	2
CO3	3	2											2	2
CO4	3	2			3				3	3			2	2
AVG	2.75	2	2		3				3	3			**2**	**2**

Parallel Computing Laboratory
[As per Choice Based Credit System (CBCS) scheme]
SEMESTER – VII

Subject Code	BCS702	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	3:0:2:0		
Total Hours of Pedagogy	40 hours Theory + 8-10 Lab slots	Total Marks	100
Credits	04	Exam Hours	02

Course Learning Objectives:

- To Provide a strong foundation in database concepts, technology, and practice.
- To Practice SQL programming through a variety of database problems.
- To Understand the relational database design principles.
- To Demonstrate the use of concurrency and transactions in database.
- To Design and build database applications for real world problems.
- To become familiar with database storage structures and access techniques.

PRACTICAL COMPONENT OF IPCC

1	Write a OpenMP program to sort an array on n elements using both sequential and parallel mergesort(using Section). Record the difference in execution time.
2	Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread. For example, if there are two threads and four iterations, the output might be the following: a. Thread 0 : Iterations 0 –– 1 b. Thread 1 : Iterations 2 –– 3
3	Write a OpenMP program to calculate n Fibonacci numbers using tasks.
4	Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.
5	Write a MPI Program to demonstration of MPI_Send and MPI_Recv.
6	Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence
7	Write a MPI Program to demonstration of Broadcast operation.
8	Write a MPI Program demonstration of MPI_Scatter and MPI_Gather

9	Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD)

Course outcomes (Course Skill Set):

At the end of the course, the student will be able to:

- Explain the need for parallel programming
- Demonstrate parallelism in MIMD system.
- Apply MPI library to parallelize the code to solve the given problem.
- Apply OpenMP pragma and directives to parallelize the code to solve the given problem
- Design a CUDA program for the given problem.

Assessment Details (both CIE and SEE) The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CIE for the theory component of the IPCC (maximum marks 50)

- IPCC means practical portion integrated with the theory of the course.
- CIE marks for the theory component are 25 marks and that for the practical component is 25 marks.
- 25 marks for the theory component are split into 15 marks for two Internal Assessment Tests (Two Tests, each of 15 Marks with 01-hour duration, are to be conducted) and 10 marks for other assessment Methods mentioned in 22OB4.2. The first test at the end of 40-50% coverage of the syllabus and the second test after covering 85-90% of the syllabus.
- Scaled-down marks of the sum of two tests and other assessment methods will be CIE marks for the theory component of IPCC (that is for 25 marks).
- The student has to secure 40% of 25 marks to qualify in the CIE of the theory component of IPCC. CIE for the practical component of the IPCC
- 15 marks for the conduction of the experiment and preparation of laboratory record, and 10 marks for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments write-ups are added and scaled down to 15 marks.
- The laboratory test (duration 02/03 hours) after completion of all the experiments shall be

CIE for the practical component of the IPCC

- 15 marks for the conduction of the experiment and preparation of laboratory record, and 10 marks for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments write-ups are added and scaled down to 15 marks.
- The laboratory test (duration 02/03 hours) after completion of all the experiments shall be conducted for 50 marks and scaled down to 10 marks.
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for 25 marks.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

Parallel Computing Laboratory Evaluation Rubrics

Subject Code: BCS702 **CIE Marks: 25**
Number of Contact Hours/Week: 2

Lab Write-up and Execution rubrics for Daily Conduction (Max: 15 Marks)

Sl. No	Task	Good	Average
a.	Understanding of problem and approach to solve (4 Marks)	Demonstrate good knowledge of design concepts of given problem and implementation. (4)	Moderate understanding of the algorithm and its implementation.(2)
b.	Execution (5 Marks)	Program has all possible conditions while execution with proper results. (5)	Average conditions are defined and verified during execution. (3)
c.	Analysis (3 Marks)	Investigate the various Algorithmic technique.(3)	Investigate a couple of Algorithmic technique.(2)
d.	Record (3 Marks)	Lab Record submitted as directed and on time (3)	Directions were not followed or submitted on time (2)

Lab Write-up and Execution rubrics for Internals (Max: 25 Marks)

Sl. No	Task	Good	Average
a.	Understanding of problem and approach to solve (6 Marks)	Demonstrate good knowledge of design concepts of given problem and implementation. (6)	Moderate understanding of the algorithm and its implementation in Java. (3)
b.	Execution (10 Marks)	Program has all possible conditions while execution with proper results. (10)	Average conditions are defined and verified during execution. (5)
c.	Analysis (4 Marks)	Investigate the various Algorithmic technique (4)	Investigate a couple of Algorithmic technique. (2)
d.	Viva (5 Marks)	Explains different steps of algorithm/problem and related concepts involved.(5)	Adequately provides explanation on algorithm/problem.(3)

Parallel Computing Laboratory

Subject Code: BCS702 **CIE Marks: 25**
Number of Contact Hours/Week: 2

Lesson Planning / Schedule of Experiments

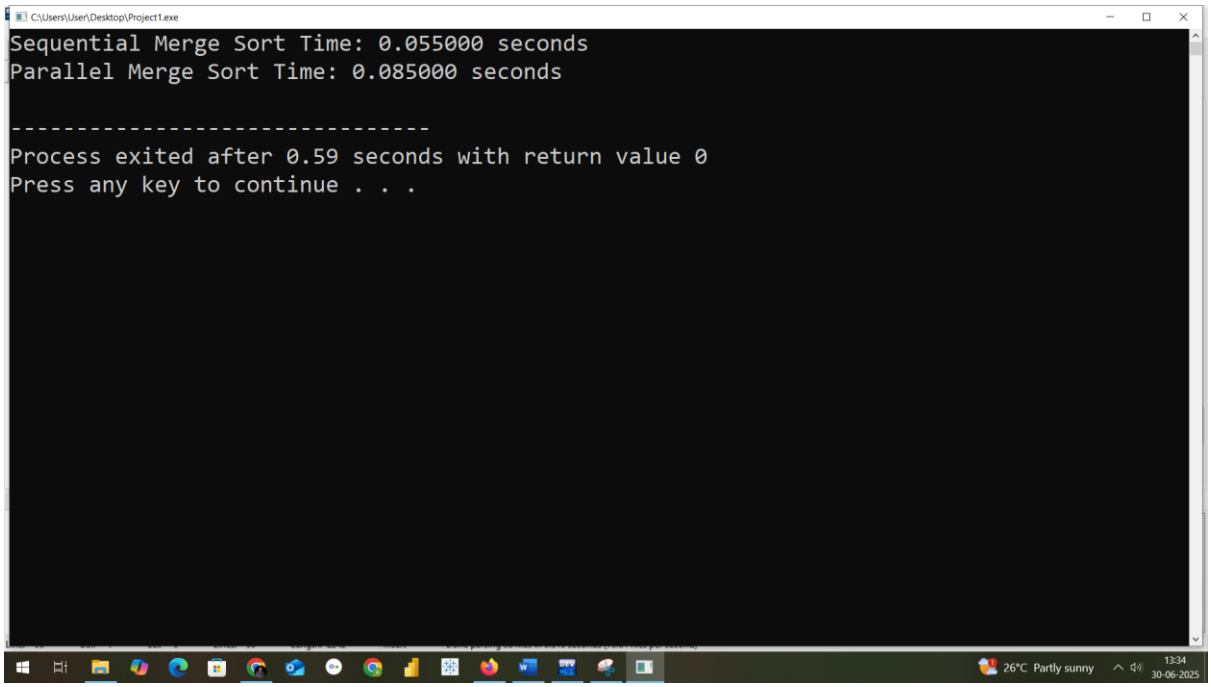
Sl. No	Name of the Experiment	To be completed
1	Sample Programs	Week 1
2	Write a Open MP program to sort an array on n elements using both sequential and parallel merge sort(using Section). Record the difference in execution time.	Week 2
3	Write an Open MP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread. For example, if there are two threads and four iterations, the output might be the following: a. Thread 0 : Iterations 0 — 1 b. Thread 1 : Iterations 2 — 3	Week 3
4	Write a Open MP program to calculate n Fibonacci numbers using tasks.	Week 4
5	Write a Open MP program to find the prime numbers from 1 to n ploying parallel for directive. Record both serial and parallel execution times.	Week 5
6	Test-I	Week 6
7	Write a MPI Program to demonstration of MPI_Send and MPI_Recv.	Week 7
8	Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence.	Week 8
9	Write a MPI Program to demonstration of Broadcast operation.	Week 9
10	Write a MPI Program demonstration of MPI_Scatter and MPI_Gather.	Week 10
11	Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD)	Week 11
12	Test-II	Week 12

Pragram 1

Write a OpenMP program to sort an array on n elements using both sequential and parallel Mergesort (using Section). Record the difference in execution time.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define SIZE 100000  // You can change this to test with larger arrays

void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    free(L);
    free(R);
}

void mergeSortSeq(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSortSeq(arr, l, m);
        mergeSortSeq(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void mergeSortParallel(int arr[], int l, int r) {
    if (r - l < SIZE){
                mergeSortSeq(arr, l, r);  // Use sequential for small problems
    } else if (l < r) {
        int m = (l + r) / 2;
        #pragma omp parallel sections
        {
```

```
 #pragma omp section
            mergeSortParallel(arr, l, m);
        }

 merge(arr, l, m, r);
   }
}
void fillArray(int arr[], int size) {
   for (int i = 0; i < size; i++)
      arr[i] = rand() % 100000;
}

void copyArray(int src[], int dest[], int size) {
   for (int i = 0; i < size; i++)
      dest[i] = src[i];
}

int main() {
   int *a1 = (int *)malloc(SIZE * sizeof(int));
   int *a2 = (int *)malloc(SIZE * sizeof(int));
   double start, end;

   fillArray(a1, SIZE);
   copyArray(a1, a2, SIZE);

   // Sequential Merge Sort
   start = omp_get_wtime();
   mergeSortSeq(a1, 0, SIZE - 1);
   end = omp_get_wtime();
   printf("Sequential Merge Sort Time: %.6f seconds\n", end - start);

   // Parallel Merge Sort
   start = omp_get_wtime();
   mergeSortParallel(a2, 0, SIZE - 1);
   end = omp_get_wtime();
   printf("Parallel Merge Sort Time: %.6f seconds\n", end - start);

   free(a1);
   free(a2);
   return 0;
}
```


C:\Users\User\Desktop\Project1.exe
Sequential Merge Sort Time: 0.055000 seconds
Parallel Merge Sort Time: 0.085000 seconds

Process exited after 0.59 seconds with return value 0
Press any key to continue . . .
26°C Partly sunny
13:34
30-06-2025

Program 2

Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread.

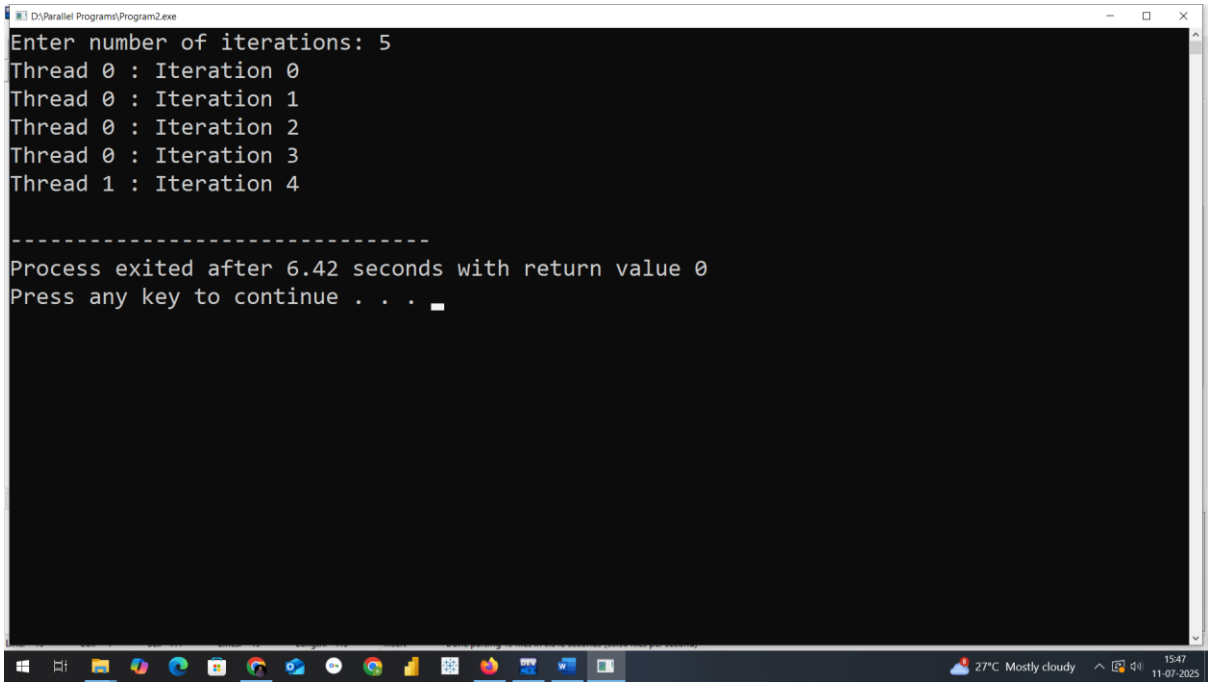
For example, if there are two threads and four iterations, the output might be the following:

a. Thread 0 : Iterations 0 –– 1

b. Thread 1 : Iterations 2 – 3

```
#include <stdio.h>
#include <omp.h>

int main() {
   int n;

   printf("Enter number of iterations: ");
   scanf("%d", &n);

   // Parallel for loop with static scheduling and chunk size 2
   #pragma omp parallel for schedule(static, 4)
   for (int i = 0; i < n; i++) {
      int tid = omp_get_thread_num();
      printf("Thread %d : Iteration %d\n", tid, i);
   }

   return 0;
}
```

Final Step Goto Execute Menu and click on Compile and Run You can see the Thread execution based on number of Iterations


D:\Parallel Programs\Program2.exe
Enter number of iterations: 5
Thread 0 : Iteration 0
Thread 0 : Iteration 1
Thread 0 : Iteration 2
Thread 0 : Iteration 3
Thread 1 : Iteration 4

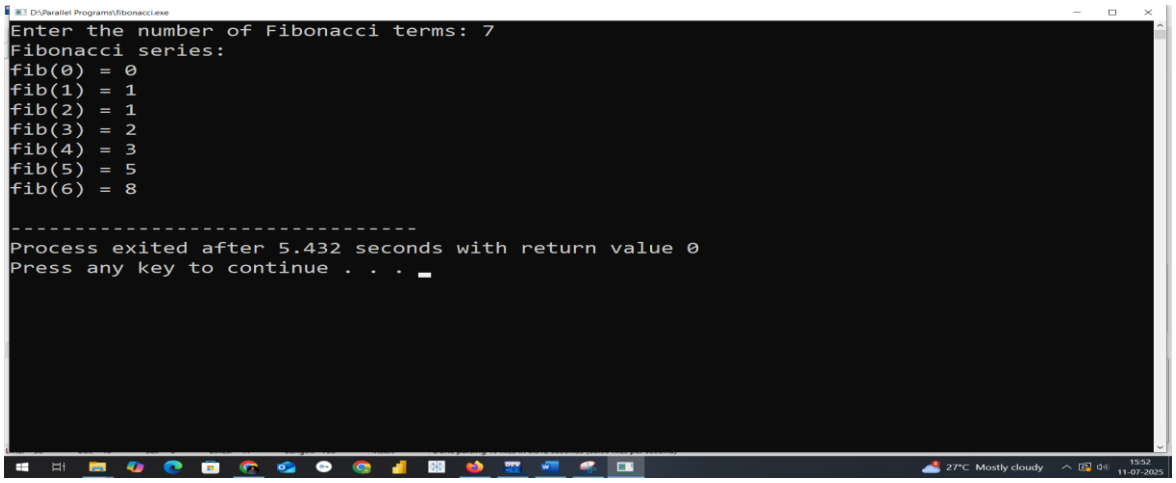
Process exited after 6.42 seconds with return value 0
Press any key to continue . . .
27°C Mostly cloudy
15:47
11-07-2025

Pragram 3

Write a OpenMP program to calculate n Fibonacci numbers using tasks.

```
#include <stdio.h>
#include <omp.h>

// Recursive Fibonacci with OpenMP Tasks
int fib_task(int n) {
   int x, y;

   if (n < 2)
      return n;

   #pragma omp task shared(x)
    x = fib_task(n - 1);

   #pragma omp task shared(y)
   y = fib_task(n - 2);

   #pragma omp taskwait
    return x + y;
}

int main() {
   int n;
   printf("Enter the number of Fibonacci terms: ");
   scanf("%d", &n);

   printf("Fibonacci series:\n");

   #pragma omp parallel
   {
      #pragma omp single
      {
         for (int i = 0; i < n; i++) {
            int result = fib_task(i);
            printf("fib(%d) = %d\n", i, result);
         }
      }
   }

   return 0;
}
```

Final Step Goto Execute Menu and click on Compile and Run You can see the results based on number of Fibonacci terms



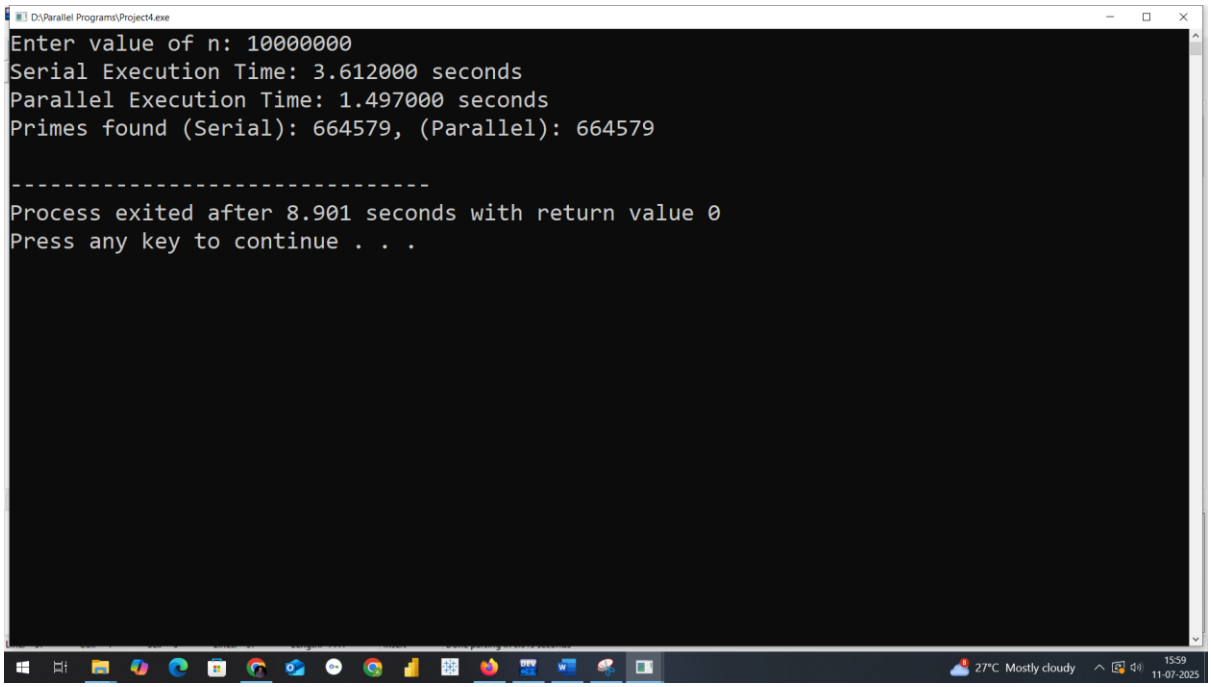
Program 4

Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <math.h>

int is_prime(int num) {
    if (num < 2) return 0;
    if (num == 2) return 1;
    if (num % 2 == 0) return 0;
    int limit = sqrt(num);
    for (int i = 3; i <= limit; i += 2)
   {
        if (num % i == 0) return 0;
    }
    return 1;
}
int main() {
    int n;
    printf("Enter value of n: ");
    scanf("%d", &n);

    double start_time, end_time;

    // Serial Execution
    start_time = omp_get_wtime();
    int serial_count = 0;
    for (int i = 2; i <= n; i++) {
        if (is_prime(i)) serial_count++;
    }
    end_time = omp_get_wtime();
    printf("Serial Execution Time: %f seconds\n", end_time - start_time);

    // Parallel Execution
    omp_set_num_threads(4); // Set based on CPU
    int* prime_flags = (int*)calloc(n + 1, sizeof(int));
    start_time = omp_get_wtime();
    #pragma omp parallel for schedule(dynamic, 100)
    for (int i = 2; i <= n; i++) {
        if (is_prime(i)) {
            prime_flags[i] = 1;
        }
    }
```

```
end_time = omp_get_wtime();

    int parallel_count = 0;
    for (int i = 2; i <= n; i++) {
        if (prime_flags[i]) parallel_count++;
    }

    printf("Parallel Execution Time: %f seconds\n", end_time - start_time);
    printf("Primes found (Serial): %d, (Parallel): %d\n", serial_count, parallel_count);

    free(prime_flags);
    return 0;
}
```

Final Step Goto Execute Menu and click on Compile and Run You can see the parallel execution is reduced more u increase the value of n.



Program 5

Write a MPI Program to demonstration of MPI_Send and MPI_Recv.

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char** argv) {
    int rank, size;
    MPI_Status status;

    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get current process rank and total number of processes
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Process 0 sends a message to all other processes
    if (rank == 0) {
        for (int i = 1; i < size; i++) {
            char message[100];
            sprintf(message, "Hello from process 0 to process %d", i);
            MPI_Send(message, strlen(message) + 1, MPI_CHAR, i, 0, MPI_COMM_WORLD);
            printf("Process 0 sent message to process %d\n", i);
        }
    } else {
        // All other processes receive the message from process 0
        char recv_buf[100];
        MPI_Recv(recv_buf, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &status);
        printf("Process %d received message: %s\n", rank, recv_buf);
    }

    // Finalize MPI
    MPI_Finalize();
    return 0;
}
```

Now For Compilation

mpicc sr.c -o sr

For execution

mpirun –oversubscribe --np 6 ./sr

```
ise@ubuntu20:~$ mpicc pp5.c -o pp5
ise@ubuntu20:~$ mpirun --oversubscribe --np 6 ./pp5
Process 0 sent message to process 1
Process 0 sent message to process 2
Process 0 sent message to process 3
Process 0 sent message to process 4
Process 0 sent message to process 5
Process 2 received message: Hello from process 0 to process 2
Process 3 received message: Hello from process 0 to process 3
Process 4 received message: Hello from process 0 to process 4
Process 1 received message: Hello from process 0 to process 1
Process 5 received message: Hello from process 0 to process 5
```

Program 6

Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence.

STEP1: Open terminal
Type

nano deadlock.c
type code as in below

```
// deadlock.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    int send_data, recv_data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        send_data = 100;
// blocking send&recieved
        MPI_Send(&send_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&recv_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 0 received %d\n", recv_data);
    } else if (rank == 1) {
        send_data = 200;
        MPI_Send(&send_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);  // blocking send
        MPI_Recv(&recv_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received %d\n", recv_data);
    }
    MPI_Finalize();
    return 0;
}
```

Now For Compilation

mpicc deadlock.c -o deadlock

For execution

mpirun --np 2 ./deadlock

```
ise@ubuntu20:~$ gedit deadlock.c
ise@ubuntu20:~$ mpicc deadlock.c -o deadlock
ise@ubuntu20:~$ mpirun -np 2 ./deadlock
Process 0 received 200
Process 1 received 100
ise@ubuntu20:~$
```

Program 7

Write a MPI Program to demonstration of Broadcast operation.
STEP1: Open terminal
Type
nano broadcast.c
type code as in below

```
// broadcast.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    int number;

    // Initialize MPI environment
    MPI_Init(&argc, &argv);

    // Get current process rank and total number of processes
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        number = 42;  // The value to be broadcasted
        printf("Process 0 is broadcasting number %d\n", number);
    }

    // Broadcast the value of `number` from process 0 to all others
    MPI_Bcast(&number, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // All processes print the received number
    printf("Process %d received number %d\n", rank, number);

    // Finalize MPI
    MPI_Finalize();
    return 0;
}
```

Now For Compilation
mpicc broadcast.c -o broadcast

For execution
mpirun –oversubscribe --np 4 ./broadcast

```
ise@ubuntu20:~$ gedit broadcast.c
ise@ubuntu20:~$ mpicc broadcast.c -o broadcast
ise@ubuntu20:~$ mpirun --oversubscribe --np 4 ./broadcast
Process 0 is broadcasting number 42
Process 0 recieved number 42
Process 1 recieved number 42
Process 2 recieved number 42
Process 3 recieved number 42
ise@ubuntu20:~$
```

Program 8

Write a MPI Program demonstration of MPI_Scatter and MPI_Gather.

STEP1: Open terminal

Type
nano sg.c
type code as in below

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    int data[100];         // Only used by root
    int recv_value;        // Value received by each process
    int gathered[100];     // Only used by root

    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get rank and size
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Only rank 0 initializes data
    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            data[i] = i * 10;
        }
        printf("Process 0 initialized data: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", data[i]);
        }
        printf("\n");
    }

    // SCATTER: One value from `data` array is sent to each process
    MPI_Scatter(data, 1, MPI_INT, &recv_value, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received value %d from Scatter\n", rank, recv_value);

    // Each process modifies its value
    recv_value += rank;
```

```
    // GATHER: Each process sends modified value back to root
      MPI_Gather(&recv_value, 1, MPI_INT, gathered, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // Only rank 0 displays gathered results
    if (rank == 0) {
        printf("Process 0 gathered data: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", gathered[i]);
        }
        printf("\n");
    }

    // Finalize
    MPI_Finalize();
    return 0;
}
```

Now For Compilation
mpicc sg.c -o sg
For execution
mpirun –oversubscribe --np 4 ./sg

```
ise@ubuntu20:~$ mpicc sg.c -o sg
ise@ubuntu20:~$ mpirun --oversubscribe --np 4 ./sg
Process 0 initialized data: 0 10 20 30
Process 0 recieved value 0 from Scatter
Process 1 recieved value 10 from Scatter
Process 2 recieved value 20 from Scatter
Process 3 recieved value 30 from Scatter
Process 0 gathered data: 0 11 22 33
```

Program 9

Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD)
STEP1: Open terminal
Type
nano pg9.c
type code as in below

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    int value;
    int sum_result, prod_result, max_result, min_result;
    int all_sum, all_prod, all_max, all_min;

    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get rank and size
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Each process sets its value
    value = rank + 1;  // Example: rank 0 = 1, rank 1 = 2, etc.
    printf("Process %d has value %d\n", rank, value);

    // ---------- MPI_Reduce (result only at root, rank 0) ----------
    MPI_Reduce(&value, &sum_result, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &prod_result, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &max_result, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &min_result, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("\n--- Results using MPI_Reduce (only on root) ---\n");
        printf("Sum   = %d\n", sum_result);
        printf("Prod  = %d\n", prod_result);
        printf("Max   = %d\n", max_result);
        printf("Min   = %d\n", min_result);
    }
```

```
// ---------- MPI_Allreduce (result sent to all processes) ----------
    MPI_Allreduce(&value, &all_sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_prod, 1, MPI_INT, MPI_PROD, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);

    printf("Process %d - AllReduce: Sum=%d Prod=%d Max=%d Min=%d\n",
           rank, all_sum, all_prod, all_max, all_min);

    // Finalize
    MPI_Finalize();
    return 0;
}
```

Now For Compilation

mpicc pg9.c -o pg9

For execution

mpirun --np 2 ./pg9

```
ise@ubuntu20:~$ mpicc pg9.c -o pg9
ise@ubuntu20:~$ mpirun --np 2 ./pg9
Process 0 has value 1

--- Results using MPI_Reduce (only on root) ---
Sum   = 3
Prod  = 2
Max   = 2
Min   = 1
Process 0 - AllReduce: Sum=3 Prod=2 Max=2 Min=1
Process 1 has value 2
Process 1 - AllReduce: Sum=3 Prod=2 Max=2 Min=1
```

Sample programs

STEP1: Download from here:

👉 WinLibs MinGW-w64 Builds

STEP2: PROCEDURE OF EXTRACTION AND SETTING PATH TO RUN OpenMP PROJECTS

1. Choose:
   - Version: **Latest GCC**
   - Architecture: x86_64
   - Threads: posix
   - Exception: seh
2. Extract it to:

C:\mingw-w64

3. Add to Path:

C:\mingw-w64\bin

STEP3: Confirm Correct Installation

In a new Command Prompt, run:
gcc -v
You should see something like:
Target: x86_64-w64-mingw32
Thread model: posix
gcc version: ...
✅ If you see **thread model: posix**, its good to go.

STEP4: Create a file omp.c through notepad:

```
#include <omp.h>
#include <stdio.h>
int main() {
   #pragma omp parallel
   {
      printf("Hello from thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads());
   }
   return 0;
}
```